Reproducible Data Analysis hw4.3

AUTHOR
Quinn Hughes

PUBLISHED
April 15, 2026

Busiest Airports Analysis

This section looks at six of the world's busiest airports: Atlanta, Frankfurt, Beijing Daxing, Dubai, London Heathrow, and Seoul Incheon. The table gives the exact passenger totals and percent changes, while the bar chart makes it easier to compare the size of the airports visually.

Table 1: Six selected airports and their passenger totals.

Rank	Airport	Country	Region	Total Passengers	YoY % Change
1	Atlanta (ATL)	United States	North America	108,067,766	3.3
2	Dubai (DXB)	United Arab Emirates	Middle East	92,331,506	6.1
5	Heathrow (LHR)	United Kingdom	Europe	83,884,572	5.9
13	Seoul Incheon (ICN)	South Korea	Asia	71,212,515	26.6
21	Frankfurt (FRA)	Germany	Europe	61,564,957	3.7
38	Beijing Daxing (PKX)	China	Asia	49,441,029	25.5

The table shows that the selected airports all handle very large numbers of passengers, but they are not equal in scale. Ranking the airports by total passengers makes the largest hubs stand out clearly. The percent change column is also useful because it shows that passenger volume and recent growth are related but not the same thing.

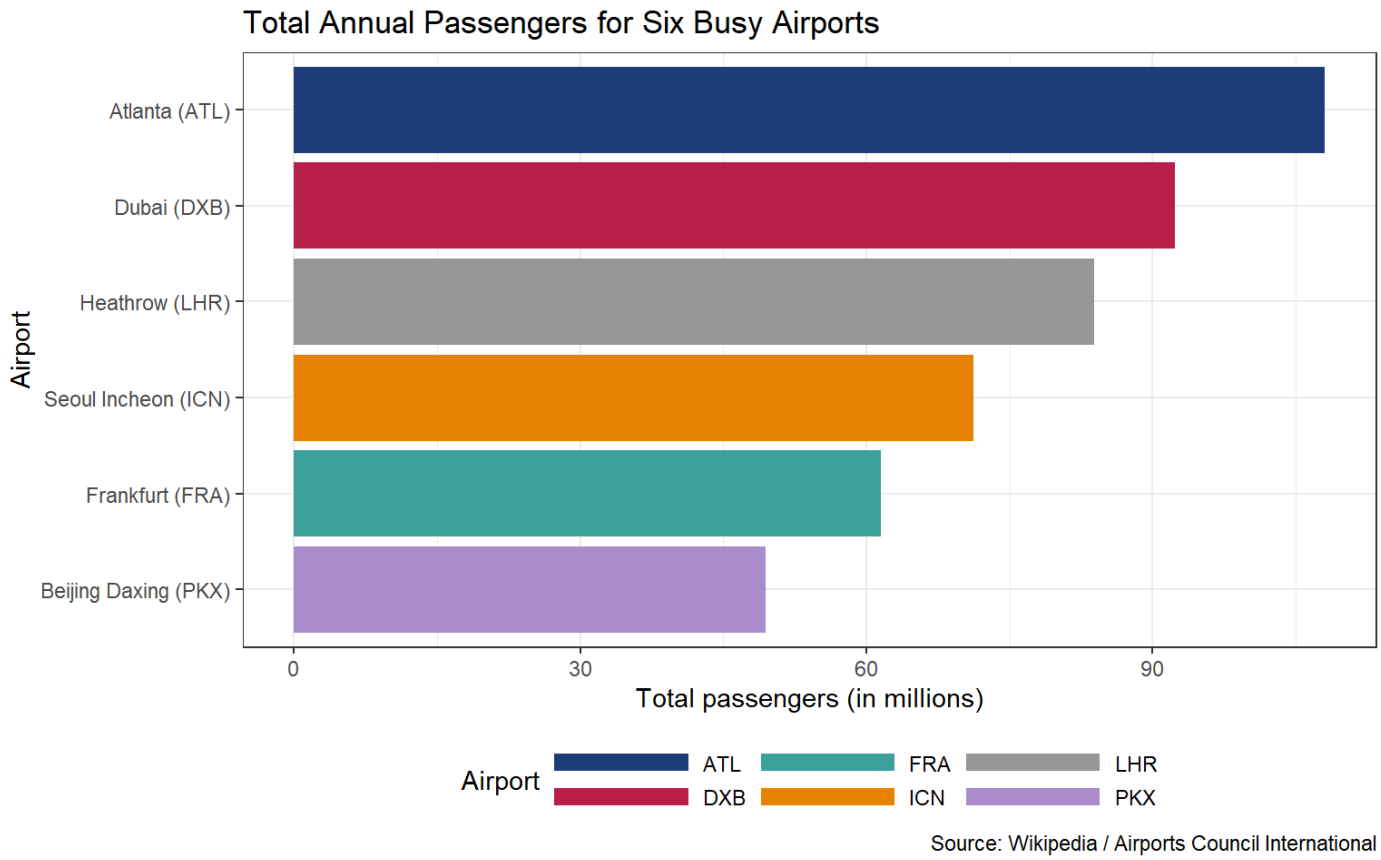

Figure 1: Passenger traffic for six selected busy airports.

The bar chart makes the comparison easier to see than the table alone. It shows the difference in passenger totals across the selected airports and also uses color to identify each airport's region. This helps show that major passenger hubs are spread across North America, Europe, Asia, and the Middle East.

Monte Carlo Numerical Integration

Monte Carlo numerical integration estimates area by randomly generating points inside a bounding box and calculating the proportion of points that fall below a curve. Here, the function is (f(x) = x^2) from 0 to 1. The exact value of the integral is (1/3), so the simulation estimates can be compared with that value.

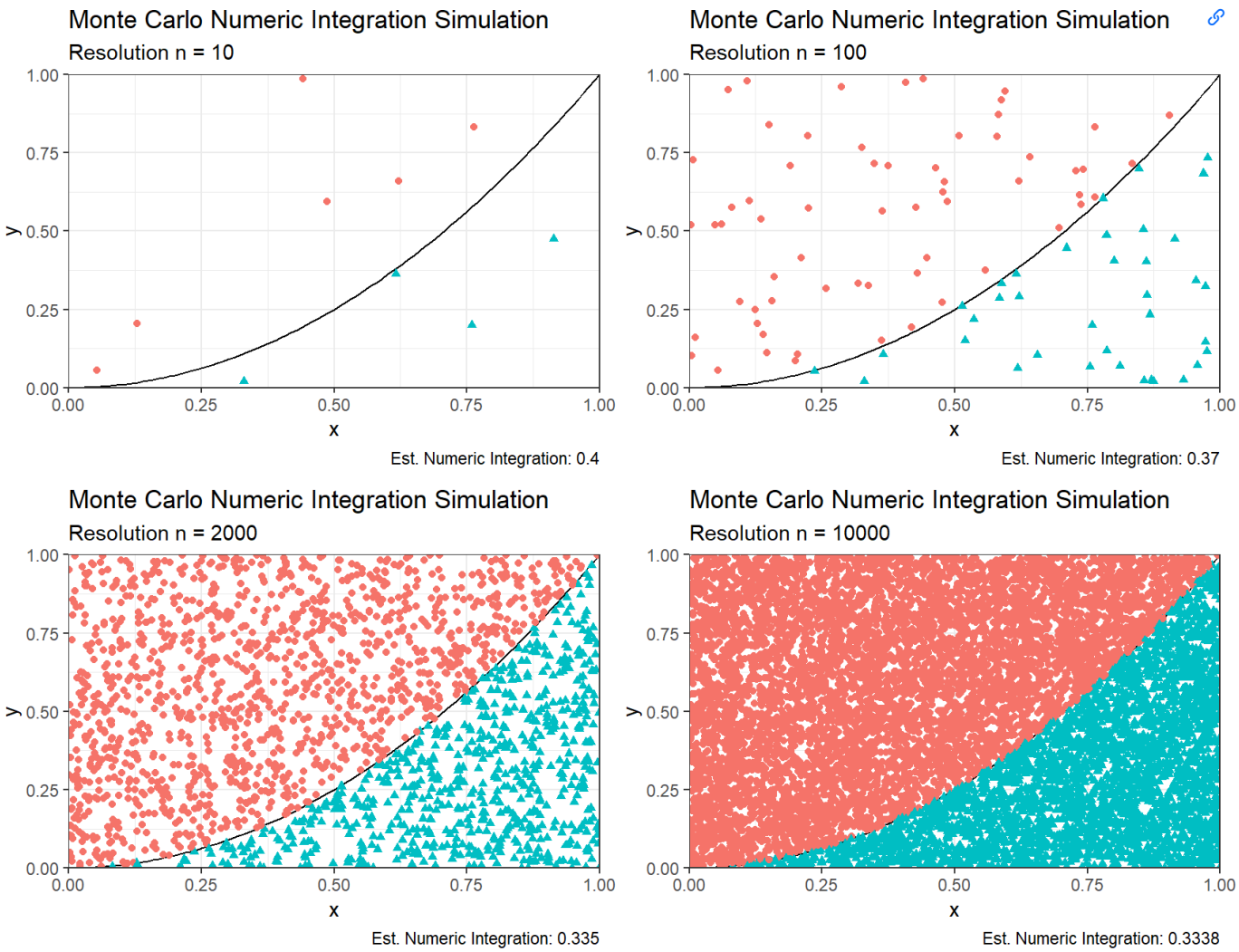

Figure 2: Monte Carlo numerical integration simulations for four sample sizes.

The small multiple visualization shows how the estimate changes as the sample size increases. With only 10 points, the estimate can be very rough because each point has a large influence on the final result. With 2,000 and 10,000 points, the random points fill the square more evenly, so the estimate should usually be closer to the true value of (1/3).

Planning and Prompting GenAI Tools

My goals and Plan

The goal of this project is to study how calcium levels change over four years for two specific groups in this dataset: 15 women in the null group and 16 women in the diet/exercise group. We want to see if the treatment helps slow calcium loss compared to the control group. To do this, we will need to load the calcium.csv file into R using read_csv() We will also create a unique ID for each woman using mutate(row_number()) so we can track individuals over time.

Next, we will reshape the data from wide format to long format so that each row represents one woman at one year. We will split the dataset into two parts (null and treatment), use pivot_longer() to convert y0–y3

into a single “year” column, and then add a “group” variable to label each observation. After that, we will combine the two datasets using bind_rows(). We will clean the “year” variable using parse_number() so that it becomes 0, 1, 2, or 3 instead of text labels. We will inspect the new dataset to make sure each participant now has four rows (one for each year) and that both groups are correctly labeled. We will also calculate summary statistics like the mean calcium level for each group and year using group_by() and summarize() to prepare for visualization.

Finally, we will create visualizations using ggplot() to compare the groups over time. First, we will make a line plot of mean calcium levels using geom_line() and geom_point() to clearly compare trends between the null and treatment groups. Then, we will create a spaghetti plot with geom_line() to show individual changes and check for variability within each group, and boxplots with geom_boxplot() to compare distributions at each year. After each plot, we will check if the results make sense (for example, whether both groups start at similar baseline levels and whether the treatment group declines more slowly). If something looks off, we will revisit earlier steps and fix the data. This plan follows the steps suggested by the AI tool (ChatGPT), which recommended reshaping the data with pivot_longer(), labeling groups, and using line plots and summary statistics to compare trends over time.

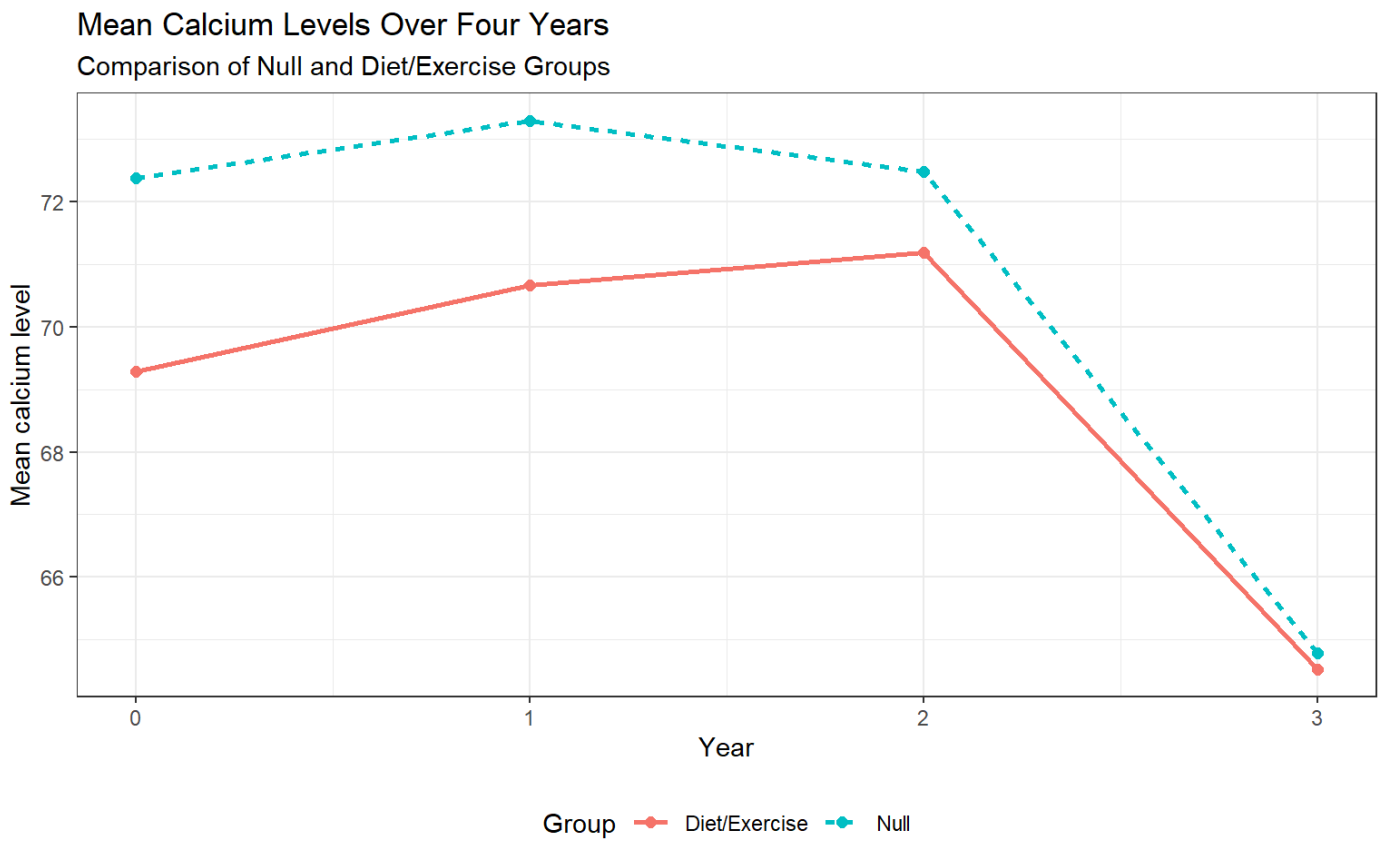

Figure 3: Mean calcium levels over four years for the null and diet/exercise groups.

Generic Plot

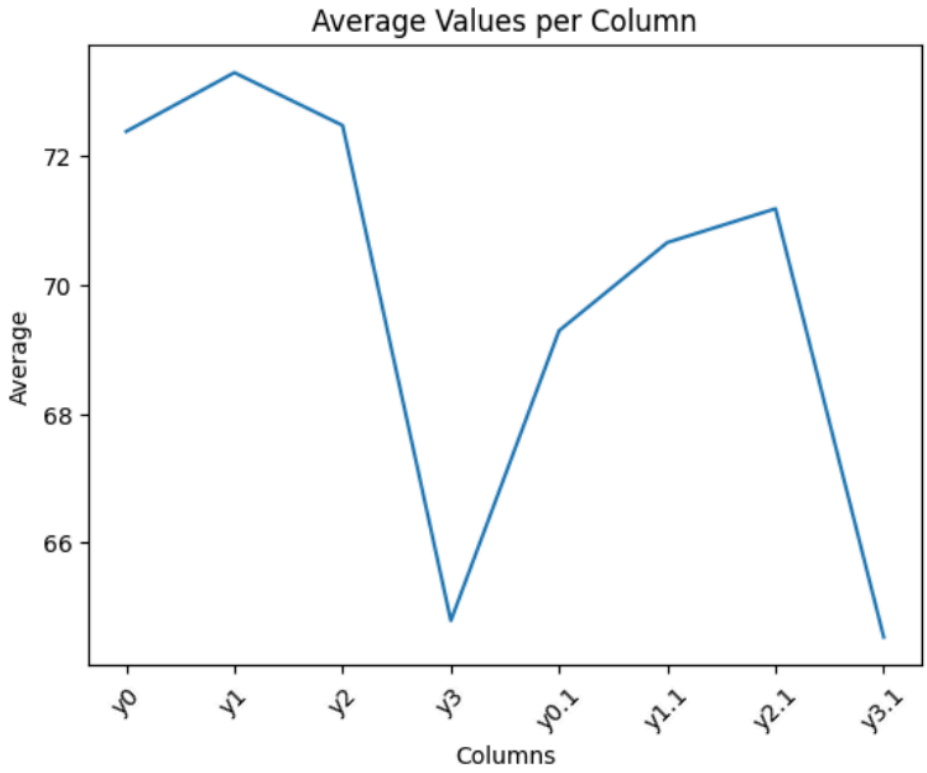

Figure 4: Generic visualization generated by AI from a generic prompt.

Generic Output

I created a visualization showing the average value of each column in your dataset.

What this shows:

- Each point represents the mean average of one column, such as y0, y1, y2, and y3.
- You can quickly compare which variables tend to have higher or lower values overall.
- Some columns cluster around 70-73, while others drop closer to 65.

Narative Compare and Contrast

The calcium plot I created is more useful because it matches the question of whether the diet/exercise treatment group appears to lose calcium more slowly than the null group over four years. The plan driven GenAI response was helpful because it focused on the structure of the data before making a graph. It recognized that the first four columns represented years 0 through 3 for the null group and that the next four columns represented years 0 through 3 for the diet/exercise group. This led it to reshape the data into long format, create a group variable, create a year variable, and then compare the two groups over time.

The plot based on that plan shows that both groups experience a decline in calcium levels over time. The null group starts around the low 70s and drops noticeably by year 3. The diet/exercise group also drops by year 3, but it is more steady during the first two years. This suggests that the treatment may help slow calcium loss compared with the null group, although the difference is not huge. In contrast, the generic visualization is much weaker because it treats the eight columns as unrelated variables. It plots the average of each column in order, using labels like y0, y1, y2, y3, y0.1, y1.1, y2.1, and y3.1. This technically shows

column averages, but it does not explain that the first four columns are the null group and the second four columns are the diet/exercise group.

My own final visualization separates the two groups and uses year as the x-axis. Each line represents one group's calcium pattern over time, which better matches the situation: women were measured repeatedly across four years, and the goal was to compare the treatment group against the null group. The generic graph is misleading because it connects all eight column averages into one line, creating the appearance of one continuous trend even though it is jumping from the null group's year 3 value to the treatment group's year 0 value. This activity shows that GenAI tools can be useful, but only when the prompt gives enough context about the data structure and the research question. A generic prompt may produce a graph that looks reasonable at first, but it can miss the meaning of the cases, groups, and time points.

Reflection

This assignment helped me see the whole course come together. Reproducible data analysis brings code, results, and writing together in one document. Instead of copying a table or graph into a separate file, Quarto allows the output to be generated directly from the code. This makes the analysis easier to update and easier for someone else to follow.

The airport analysis showed the importance of cleaning data before making a table or graph. The original data came from a web page, so the columns had to be renamed, numeric values had to be parsed, and the selected airport codes had to be extracted. Those steps made it possible to create a cleaner table and a more readable visualization. We've done these exact things in previous assignments and in learning how to clean the data after scraping.

The Monte Carlo section helped me understand simulation as a way to approximate a mathematical result. The same method was repeated with different sample sizes, and the plots showed why larger samples usually produce better estimates. This made the idea of numerical integration more visual and concrete. We've worked with this more recently and have been getting better at using real statistical methods to crate inferences from data.

The calcuim data took us back to some of the earliest concepts of the course. First it was a reminder of the importance of tidying data and how that is done by pivoting to make it so there is a case per row. Then how once we have tidy data we can make better observations.

I also learned that GenAI tools are more useful when they are given a specific plan. A vague prompt may produce a general answer, but a prompt that describes the data, goal, and expected output is more likely to produce code that fits the actual task.

GenAI Usage Appendix

Tool Details

- I used ChatGPT
- Version information: GPT-5.3
- Date of usage: 4/16/2026

Prompt

The goal of this project is to study how calcium levels change over four years for two specific groups in this dataset: 15 women in the null group and 16 women in the diet/exercise group. We want to see if the treatment helps slow calcium loss compared to the control group. To do this, we will need to load the calcium.csv file into R using read_csv() We will also create a unique ID for each woman using mutate(row_number()) so we can track individuals over time.

Next, we will reshape the data from wide format to long format so that each row represents one woman at one year. We will split the dataset into two parts (null and treatment), use pivot_longer() to convert y0–y3 into a single "year" column, and then add a "group" variable to label each observation. After that, we will combine the two datasets using bind_rows(). We will clean the "year" variable using parse_number() so that it becomes 0, 1, 2, or 3 instead of text labels. We will inspect the new dataset to make sure each participant now has four rows (one for each year) and that both groups are correctly labeled. We will also calculate summary statistics like the mean calcium level for each group and year using group_by() and summarize() to prepare for visualization.

Finally, we will create visualizations using ggplot() to compare the groups over time. First, we will make a line plot of mean calcium levels using geom_line() and geom_point() to clearly compare trends between the null and treatment groups. Then, we will create a spaghetti plot with geom_line() to show individual changes and check for variability within each group, and boxplots with geom_boxplot() to compare distributions at each year. After each plot, we will check if the results make sense (for example, whether both groups start at similar baseline levels and whether the treatment group declines more slowly). If something looks off, we will revisit earlier steps and fix the data. This plan follows the steps suggested by the AI tool (ChatGPT), which recommended reshaping the data with pivot_longer(), labeling groups, and using line plots and summary statistics to compare trends over time.

Response 1

```
library(tidyverse)

# Step 1: Load the calcium data
calciumRaw <- read_csv(
```

```r
  "C:/Users/quinn/Downloads/calcium.csv",
  show_col_types = FALSE,
  name_repair = "unique"
)

# Rename columns so the null and treatment groups are clear
names(calciumRaw) <- c(
  "null_y0", "null_y1", "null_y2", "null_y3",
  "treatment_y0", "treatment_y1", "treatment_y2", "treatment_y3"
)

# Step 2: Reshape the null group from wide to long format
nullData <- calciumRaw |>
  select(starts_with("null_")) |>
  filter(if_any(everything(), ~ !is.na(.x))) |>
  mutate(
    id = paste0("N", row_number()),
    group = "Null"
  ) |>
  pivot_longer(
    cols = starts_with("null_"),
    names_to = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = parse_number(year)
  )

# Step 3: Reshape the treatment group from wide to long format
treatmentData <- calciumRaw |>
  select(starts_with("treatment_")) |>
  filter(if_any(everything(), ~ !is.na(.x))) |>
  mutate(
    id = paste0("T", row_number()),
    group = "Diet/Exercise"
  ) |>
  pivot_longer(
    cols = starts_with("treatment_"),
    names_to = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = parse_number(year)
  )

# Step 4: Combine both groups into one tidy dataset
calciumLong <- bind_rows(nullData, treatmentData)

# Step 5: Calculate mean calcium level by group and year
calciumSummary <- calciumLong |>
  group_by(group, year) |>
```

```
  summarize(
    mean_calcium = mean(calcium, na.rm = TRUE),
    .groups = "drop"
  )

# Step 6: Create the main visualization comparing group means over time
calciumSummary |>
  ggplot(
    mapping = aes(
      x = year,
      y = mean_calcium,
      color = group,
      linetype = group
    )
  ) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(
    breaks = 0:3
  ) +
  labs(
    title = "Mean Calcium Levels Over Four Years",
    subtitle = "Comparison of Null and Diet/Exercise Groups",
    x = "Year",
    y = "Mean calcium level",
    color = "Group",
    linetype = "Group",
    alt = "Line graph comparing average calcium levels over four years for the null group and the
  ) +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
```

Follow-Up Prompts and Responses

No follow-up prompts were used.

Code Appendix

```r
# ============================================================
# Setup



library(tidyverse)
library(rvest)
library(knitr)
library(kableExtra)
library(ggplot2)
library(dplyr)
library(patchwork)

options(knitr.kable.NA = "")




# Busiest Airports Analysis
# ============================================================

# Step 1: Scrape the busiest airports page
airportRawList <- read_html(
  "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |>
  html_elements(css = "table") |>
  html_table()

# Step 2: Extract the target table
airportsRaw <- airportRawList[[2]]

# Step 3: Clean column names and convert numeric columns
airportsClean <- airportsRaw |>
  rename(
    Rank             = 1,
    Airport          = 2,
    Location         = 3,
    Country          = 4,
    IATA_ICAO        = 5,
    Total_Passengers = 6,
    Rank_Change      = 7,
    Pct_Change       = 8
  ) |>
  filter(!is.na(Rank), Rank != "Rank") |>
  mutate(
    Total_Passengers = parse_number(Total_Passengers),
    Pct_Change       = parse_number(Pct_Change)
  )
```

```r
# Step 4: Keep the six selected airports
targetCodes <- c("ATL", "FRA", "PKX", "DXB", "LHR", "ICN")

airportsSubset <- airportsClean |>
  mutate(IATA = str_extract(IATA_ICAO, "^[A-Z]{3}")) |>
  filter(IATA %in% targetCodes) |>
  mutate(
    Label = case_when(
      IATA == "ATL" ~ "Atlanta (ATL)",
      IATA == "FRA" ~ "Frankfurt (FRA)",
      IATA == "PKX" ~ "Beijing Daxing (PKX)",
      IATA == "DXB" ~ "Dubai (DXB)",
      IATA == "LHR" ~ "Heathrow (LHR)",
      IATA == "ICN" ~ "Seoul Incheon (ICN)"
    ),
    Region = case_when(
      IATA == "ATL"             ~ "North America",
      IATA %in% c("FRA", "LHR") ~ "Europe",
      IATA %in% c("PKX", "ICN") ~ "Asia",
      IATA == "DXB"             ~ "Middle East"
    )
  )

# Step 5: Create the formatted airport table
airportsSubset |>
  arrange(desc(Total_Passengers)) |>
  select(Rank, Label, Country, Region, Total_Passengers, Pct_Change) |>
  rename(
    Airport            = Label,
    "Total Passengers" = Total_Passengers,
    "YoY % Change"     = Pct_Change
  ) |>
  kable(
    align       = "clllrc",
    format.args = list(big.mark = ",")
  ) |>
  kable_classic() |>
  row_spec(
    row        = c(1, 3, 5),
    background = "#FFFFC5"
  ) |>
  footnote(
    general = paste(
      "Highlighted rows indicate selected airports of interest.",
      "YoY % Change is relative to the prior year."
    ),
    footnote_as_chunk = TRUE
  )

# Step 6: Create the airport passenger traffic plot
```

```r
Palette <- c(
  "#1E407C", "#BC204B", "#3EA39E", "#E98300",
  "#999999", "#AC8DCE", "#F2665E", "#99CC00"
)

airportsSubset |>
  mutate(
    passengers_mil = Total_Passengers / 1000000
  ) |>
  ggplot(
    mapping = aes(
      x = reorder(Label, passengers_mil),
      y = passengers_mil,
      fill = IATA
    )
  ) +
  geom_col() +
  coord_flip() +
  theme_bw() +
  labs(
    title   = "Total Annual Passengers for Six Busy Airports",
    x       = "Airport",
    y       = "Total passengers (in millions)",
    fill    = "Airport",
    caption = "Source: Wikipedia / Airports Council International",
    alt     = paste(
      "Bar graph comparing total annual passenger traffic",
      "for six busy airports."
    )
  ) +
  scale_fill_manual(values = Palette) +
  theme(
    legend.position   = "bottom",
    legend.key.width  = grid::unit(2, "cm"),
    legend.key.height = grid::unit(0.25, "cm")
  )



# Monte Carlo Numerical Integration
# ==========================================================

# Step 1: Generate random points inside the bounding box
set.seed(184)

myPoints <- tibble(
  x = runif(n = 10000, min = 0, max = 1),
  y = runif(n = 10000, min = 0, max = 1)
)

# Step 2: Flag points above or on/below the curve y = x^2
```

```r
myPoints <- myPoints |>
  mutate(
    flag = case_when(
      y > x^2 ~ "above",
      TRUE    ~ "on/below"
    ),
    numFlag = case_when(
      y > x^2 ~ 0,
      TRUE    ~ 1
    ),
    case   = row_number(),
    prop   = cumsum(numFlag) / case,
    numInt = prop * (1 - 0) * (1 - 0)
  )

# Step 3: Create a function for one Monte Carlo plot
make_mc_plot <- function(n_points) {
  myPoints |>
    slice_head(n = n_points) |>
    ggplot(
      mapping = aes(
        x     = x,
        y     = y,
        shape = flag,
        color = flag
      )
    ) +
    stat_function(
      fun   = function(x) x^2,
      xlim  = c(0, 1),
      color = "black"
    ) +
    geom_point() +
    scale_x_continuous(limits = c(0, 1), expand = expansion()) +
    scale_y_continuous(limits = c(0, 1), expand = expansion()) +
    labs(
      title = "Monte Carlo Numeric Integration Simulation",
      subtitle = paste("Resolution n =", n_points),
      caption = paste(
        "Est. Numeric Integration:",
        round(myPoints$numInt[n_points], digits = 4)
      ),
      x     = "x",
      y     = "y",
      shape = "Point location",
      color = "Point location",
      alt   = paste(
        "Monte Carlo scatterplot with",
        n_points,
        "random points showing whether points are above",
        "or on or below the curve y equals x squared."
```

```r
      )
    ) +
    theme_bw() +
    theme(legend.position = "none")
}

# Step 4: Build four plots and combine them
plot10    <- make_mc_plot(10)
plot100   <- make_mc_plot(100)
plot2000  <- make_mc_plot(2000)
plot10000 <- make_mc_plot(10000)

plot10 + plot100 + plot2000 + plot10000



# Calcium Data Visualization
# ==========================================================

# Step 1: Load the calcium data
calciumRaw <- read_csv(
  "C:/Users/quinn/Downloads/calcium.csv",
  show_col_types = FALSE,
  name_repair = "unique"
)

# Step 2: Rename columns to identify group and year
names(calciumRaw) <- c(
  "null_y0", "null_y1", "null_y2", "null_y3",
  "treatment_y0", "treatment_y1", "treatment_y2", "treatment_y3"
)

# Step 3: Reshape the null group from wide to long format
nullData <- calciumRaw |>
  select(starts_with("null_")) |>
  filter(if_any(everything(), ~ !is.na(.x))) |>
  mutate(
    id    = paste0("N", row_number()),
    group = "Null"
  ) |>
  pivot_longer(
    cols      = starts_with("null_"),
    names_to  = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = parse_number(year)
  )

# Step 4: Reshape the diet/exercise group from wide to long format
treatmentData <- calciumRaw |>
```

```r
  select(starts_with("treatment_")) |>
  filter(if_any(everything(), ~ !is.na(.x))) |>
  mutate(
    id    = paste0("T", row_number()),
    group = "Diet/Exercise"
  ) |>
  pivot_longer(
    cols      = starts_with("treatment_"),
    names_to  = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = parse_number(year)
  )

# Step 5: Combine both groups into one tidy dataset
calciumLong <- bind_rows(nullData, treatmentData)

# Step 6: Calculate mean calcium level by group and year
calciumSummary <- calciumLong |>
  group_by(group, year) |>
  summarize(
    mean_calcium = mean(calcium, na.rm = TRUE),
    .groups = "drop"
  )

# Step 7: Plot mean calcium levels over time
calciumSummary |>
  ggplot(
    mapping = aes(
      x        = year,
      y        = mean_calcium,
      color    = group,
      linetype = group
    )
  ) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(
    breaks = 0:3
  ) +
  labs(
    title    = "Mean Calcium Levels Over Four Years",
    subtitle = "Comparison of Null and Diet/Exercise Groups",
    x        = "Year",
    y        = "Mean calcium level",
    color    = "Group",
    linetype = "Group",
    alt      = paste(
      "Line graph comparing average calcium levels over four years",
      "for the null group and the diet exercise group."
```

```
    )
  ) +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
```